

Step 3 — Describe the system

Use this guide alongside **Step 3** of the HTML worksheet. You enter the pieces and arrows; the worksheet generates your diagram source.

This is a suggested model for a small scope: **a customer searching for products and viewing product details**. It does not claim to cover every Juice Shop feature. We have verified that the local application responds, but have not verified your browser actions or its internal database design. Confirm the visible behavior yourself and identify the internal details below as assumptions.

1. First, observe the two activities you will model

In your local Juice Shop:

1. Search for a product using the search control.
2. Open a product and read its displayed details.
3. Note what you entered and what appeared: for example, search text, product names, prices, descriptions, or reviews. Include only the fields you actually see.

If you also want to model registration, login, or ordering, add those flows after inspecting them. You do not need to pretend you verified a feature just because it appears in the starter model.

2. Enter the pieces

The following is a manageable starting model for the browsing scope. Edit the worksheet's existing rows and use **Add a piece** or the remove button as needed.

Name to enter	Type to select	Zone to enter	Basis and limitation
Customer	External entity	User's browser	The person using the application. For this simplified model, the person and their browser belong to the same customer-controlled zone.
Browser interface	Process	User's browser	The interface running in the browser: it accepts actions and displays results.
Web application	Process	Application server	The running Juice Shop application responding at <code>localhost : 3000</code> . This is a logical server-side component.
Product database	Data store	Data store	A provisional logical store for product information. Browsing alone does not establish its implementation, storage location, or access controls.

How to change the starter rows

1. Keep **Customer** and **Web application**.
2. Keep **Product database**, but record its assumed status in your notes.
3. Remove **Login service** from this browsing-only model. This does not assert that login is absent; it is outside this initial scope.
4. Remove **Uploaded files** from this browsing-only model. Do not invent an upload flow you have not observed.
5. Add **Browser interface** with the type and zone shown above.
6. Remove or replace the starter arrows. Deleting a piece does not necessarily remove its old arrows, so check every arrow's endpoints.

What the zones mean: these are logical trust/control groupings. The database is not necessarily on a separate machine or in a separate container. Treat the application-to-store division as a proposed boundary whose enforcement still needs verification. The browser and server can be on your laptop while still having different responsibilities and levels of trust.

3. Enter the arrows

Use **Add an arrow** to create the following flows. Choose the exact piece names from the From and To menus.

From	To	Suggested text for “Which data”	What you must check
Customer	Browser interface	Search text and selected product	Confirm that you searched and selected an item.
Browser interface	Customer	Product names, prices, and descriptions	Adjust the list to the information actually displayed.
Browser interface	Web application	Product search request	Check the browser's Network panel if you want to establish which request carries the search text.
Web application	Browser interface	Product search results	Confirm the response in the Network panel; otherwise label this as a logical flow inferred from the displayed results.
Web application	Product database	Product lookup criteria	Assumption about internal processing; not established by browser inspection.
Product database	Web application	Matching product records	Assumption about the source of returned product data; not established by browser inspection.

These labels describe a proposed logical data flow, not a verified sequence of separate network requests. In particular, opening a product might reuse data already loaded by the browser. **Do not add a separate “product details request” unless you observe one.**

If you cannot verify the server-side search request yet, retain the model as a draft and explicitly note that assumption. If inspection shows the browser filters already-loaded products, revise the arrows to describe product loading instead of server-side search.

4. Optional: verify the browser/server arrows

This is a read-only observation exercise; no attack or scanner is needed.

1. Open your browser's developer tools using its menu and select **Network**.
2. Leave the panel open and search for a product again.
3. Inspect the new requests. Filtering to **Fetch/XHR**, if available, may make them easier to find.
4. Record the request method, path, and where the search text appears, if present.
5. Inspect the response and note whether it includes the products you see on screen.
6. Open a product and see whether this produces a new request or uses existing data.

Record only what you observed. You do not need to copy authentication values or other sensitive request contents into your assignment.

5. Write a scope and assumptions note

The worksheet does not provide a dedicated scope field. Add your notes to `observation-notes.md` in this folder and use them when explaining confidence in Step 4 and evidence in Step 8.

Use this scaffold, replacing the brackets with your own observations:

Scope: This version models a customer searching the product catalog and viewing product information. Registration, login, checkout, uploads, and administrative features are outside this initial scope.

Observed: I searched for [term] and saw [actual result]. I opened [product] and saw [actual fields].

Browser/server evidence: [request and response inspected, or state that the browser/server flow remains inferred].

Assumption: Product information is represented as a logical product store. I have not verified the database implementation or its access controls.

Boundary uncertainty: The separate Data store zone expresses a proposed logical trust division, not evidence of a separate host or an enforced security boundary.

Do not claim a narrow model covers the whole application. If your instructor expects a broader scope, or your later abuse cases involve accounts, orders, or uploads, expand the model to include those activities and their evidence before final submission.

6. Check what the diagram should show

With the exact four pieces and six arrows above:

- There are **four elements** in **three zones**.
- Two arrows stay within the customer's zone.
- Four arrows cross zones.
- The worksheet should derive **four directional boundary entries**: browser → application, application → browser, application → store, and store → application.

The untouched starter produces three boundary entries. Your count can differ because you changed the model. A count is a consistency check, not a target to optimize for.

Check every arrow has valid endpoints and a specific data label. Look for abandoned arrows pointing at a deleted piece. Read the picture as a story: customer acts, interface requests or uses data, application returns information, customer sees results.

7. Export and save your work

1. Click **Download dfd-v1.0.mmd** in Step 3.

2. Save or move that file into this folder:

```
``text Threat Model LAB-002 v2.1/ DevSecOps-Intro/ assurance/ week02/
...

```

3. Click **Copy the diagram text**, then **Open the viewer**.

4. Paste the source into the viewer's editor and inspect the diagram.

5. Export a PNG using the viewer's export controls. Save it here as `dfd-v1.0.png`.

6. Confirm the PNG and `.mmd` show the same current model.

7. Return to Step 2 and download a fresh `threat-model-session.json` backup.

If the viewer is unavailable, the lab permits a hand-drawn diagram saved as a PNG, together with the generated `.mmd` source.

8. Before moving to Step 4

☐ I performed the activities included in my scope.

☐ I entered the pieces, their types, and their zones.

☐ I replaced the starter arrows with flows appropriate to my scope.

☐ I distinguished observations from assumptions in my notes.

☐ I checked the diagram for disconnected or deleted endpoints.

☐ I saved both `dfd-v1.0.mmd` and `dfd-v1.0.png` in `assurance/week02/`.

☐ I downloaded an updated session backup.

Next, **Step 4** asks why each derived crossing is a real trust boundary and how confident you are. The worksheet's automatic listing identifies zone changes; your explanation must establish why those changes matter.